

Adjacency 2 — CKS Is Not Parametric Memory: Standalone Treatment of the Architectural Boundary Between CKS and In-Weight Memory

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 5 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the boundary between CKS and parametric memory as a standalone architectural specification, so that downstream work can adopt, compose, or argue against the pattern without conflating its substrate commitment with in-weight storage.

Abstract

The Coordination Knowledge Substrate (CKS) pattern is most often conflated with three adjacent design objects: retrieval-augmented generation (RAG), parametric (in-weight) memory, and external structured memory in the style of Knowledge Objects (KO) and OIDA. The parent foundational note (A1.14) committed to the three adjacencies; the integrating-frame note (A2.81) named each at the integrating level; sibling notes A2.82 (RAG) and A2.84 (KO/OIDA) treat the other two as standalone specifications. This note formalizes the second adjacency — CKS is not parametric memory — as standalone, focusing on the architectural distinction between external-substrate and in-weight storage. It states what parametric memory is precisely, names the four operational components of the CKS-distinguishing commitment (storage location, expansion mechanism, retrieval mechanism, cost-curve dynamics), distinguishes the boundary from four adjacent in-weight patterns (continual learning, knowledge editing, fine-tuning with retrieval, hybrid parametric–non-parametric architectures), names the failure modes that violate the boundary, and provides an operational test for whether a system's architectural posture distinguishes it from parametric memory.

1. Why the boundary needs to be formalized as standalone

The parent foundational note A1.14 commits to three adjacencies. The integrating-frame note A2.81 named the CKS-vs-parametric-memory boundary at the integrating level; A2.82 formalizes Adjacency 1 (RAG). This note formalizes Adjacency 2 as having independent architectural content with particular weight on the storage-location and cost-curve distinctions.

The motivating cases are deployments and analyses where CKS is conflated with parametric memory: analyses that treat "memory" in CKS as equivalent to "memory" in continual learning, importing cost-curve assumptions the architecture does not have; deployments that treat CKS expansion as fine-tuning expansion, missing the architectural distinction in expansion mechanism; evaluations that

treat CKS retrieval as parametric recall, missing the determinism commitment per A2.57.

A second motivation is strategic. The boundary is consequential prior art because parametric memory is the foundational LLM concern — extensive prior work in continual learning, knowledge editing, and parametric expansion exists. Patentable derivations focused on AI-coordination architectures with external substrate, determinism-coherent AI memory, or linear-cost AI knowledge systems are substantially more defensibly contested when the boundary is publicly formalized as standalone.

A third motivation is the connection to A1.06 (linear-cost) and A2.57 (Guarantee A: read determinism). The boundary specifies the architectural commitments A1.06 and A2.57 operationalize; without it, those commitments are ambiguous because audiences import parametric assumptions into CKS expectations. The boundary is also load-bearing for A1.02, A1.04's Property C per A2.21, A1.07's provenance architecture per A2.40, and A1.16's hybrid composition framework per A2.85.

2. Parametric memory, defined precisely

Parametric memory is the design pattern in which knowledge is stored within the model's parameters — encoded into the LLM's weights through training procedures, recovered through forward passes over weight-encoded content. The pattern has four characteristic properties.

Storage location. Knowledge lives inside the LLM's parameters. The architectural pattern is in-weight memory: the model's weights are the storage medium.

Expansion mechanism. Expansion happens through training procedures that modify model weights. Continual learning, fine-tuning, instruction tuning, and knowledge editing all operate on parametric memory through gradient updates; new knowledge is encoded into weights via training, and the model itself changes.

Retrieval mechanism. Retrieval is forward passes over weight-encoded content. The LLM accesses parametric memory by generating tokens whose distribution is shaped by weight-encoded knowledge — there is no separate retrieval architecture; the knowledge is implicit in the model's generation.

Cost-curve dynamics. Empirically, parametric expansion exhibits superlinear cost dynamics: adding new knowledge through training requires substantial compute, with marginal cost growing with model size and content scale (§6.2 of the source paper). The cost curve is not linear-in-additions.

Parametric memory addresses fluency, capability acquisition, and domain knowledge — the LLM "knows" the content because the content is in the weights.

3. The CKS-distinguishing commitment

CKS distinguishes itself from parametric memory along four operational components, each corresponding to a different axis on which the architectures diverge.

(a) Different storage location. The CKS substrate is external to the LLM. The substrate persists in storage architecture per A2.08 — file systems, databases, structured stores — distinct from the LLM's weights. Per A1.02 (substrate-cell boundary), the substrate is architecturally separate from cells; per A2.21 (Property C of the AI-as-substrate-mediator decomposition), the LLM does not maintain substrate-relevant content in weights or in cell-internal state.

(b) Different expansion mechanism. Substrate state changes through human-governed writes per A1.01. The labor of expansion is allocable across three modes per A1.12: humans authoring directly per A2.69 (Mode 1), LLMs in cells writing under orchestration rules per A2.70 (Mode 2), and stable cells operating under refined rules per A2.71 (Mode 3). What expansion is *not* is gradient updates to the model: the substrate grows by writes; the model does not change as the substrate grows.

(c) Different retrieval mechanism. Substrate is recovered through deterministic reads per Guarantee A per A2.57 — the same substrate state produces the same content deterministically across operations. Reads use addressable identifiers per Guarantee C per A2.59 and substrate-only paths per A2.41. Parametric memory's recovery through forward passes is operationally distinct: non-deterministic, subject to forgetting, sensitive to prompt phrasing.

(d) Different cost-curve dynamics. CKS expansion is linear-in-additions per A1.06 (§6.1) and the cost decomposition A2.29–A2.34: storage cost scales linearly with content per A2.30; processing cost scales with operations performed, not with substrate size, per A2.32. Parametric expansion is superlinear and frontier-bending per §6.2.

The four components together define the boundary architecturally. A system that satisfies all four has the boundary in the architectural sense; the components are independent, and only their conjunction constitutes it.

4. What the boundary does NOT claim

The standalone treatment is bounded. Five things it does not claim:

It does not claim parametric memory is inferior to CKS. Parametric memory addresses different design goals — fluency, capability acquisition, domain knowledge. The boundary specifies architectural distinction, not superiority.

It does not foreclose CKS-with-fine-tuned-mediator hybrid compositions. Per A1.16 and A2.85, CKS may compose with fine-tuned LLMs in hybrid systems where the mediator's parametric memory improves fluency in a domain while the substrate remains authoritative for what has been decided in that domain. The boundary specifies what each layer is for; the hybrid uses both layers explicitly.

It does not require CKS to operate on stateless LLMs. LLMs may have parametric memory from prior training. The architectural commitment is that the LLM's parametric memory is not what makes the system CKS. CKS commitments operate on the substrate-layer architecture, not on the LLM's parametric properties.

It does not specify implementation patterns for substrate storage. Implementations may use various storage technologies for substrate — relational stores, document stores, version-control systems — provided the four components per §3 are operationally satisfied.

It does not foreclose fine-tuned mediators in CKS cells. Per A1.04 and the mediator decomposition A2.18–A2.23, the LLM in a cell may have fine-tuning applied to improve its operation under specific orchestration rules. The architectural commitment is that the mediator role is preserved (the five mediator properties hold) regardless of whether the LLM has fine-tuned parametric memory; implementations may also use indexing, caching, or other performance optimizations on substrate access, provided the four components per §3 hold.

5. What the boundary is NOT — four adjacent in-weight patterns

Four parametric variations are commonly conflated with CKS. Each is a real and reasonable design object in some other architecture; conflating any with CKS misreads the substrate's architectural commitment.

Not continual learning. Continual learning is the operational pattern of progressively updating model weights through ongoing training as new knowledge becomes available. CKS commits to external substrate with linear-cost expansion through human-governed writes per A1.01; continual learning operates through weight updates with superlinear cost dynamics per §6.2. Continual learning addresses model capability over time; CKS addresses coordination over time.

Not knowledge editing. Knowledge editing is the operational pattern of selectively modifying model weights to update specific facts or behaviors without full retraining. Knowledge editing operates on parametric memory, regardless of how surgically it modifies weights; CKS commits to substrate, regardless of how the storage backend is implemented.

Not fine-tuning with retrieval. Fine-tuning with retrieval combines parametric memory (fine-tuning) with non-parametric memory (retrieval, often RAG-style). CKS commits to substrate as a coordination artifact, distinct from both layers. A CKS hybrid may include fine-tuned mediators and RAG-style grounding — per A2.82 (the RAG boundary) and A2.85 (hybrid composition coherence) — while remaining CKS at the substrate layer.

Not hybrid parametric–non-parametric architectures. Hybrid parametric–non-parametric architectures combine in-weight memory with external memory in various configurations. CKS substrate is a specific kind of external memory committed to coordination content under human governance per A1.01; not all external memory in hybrid architectures qualifies as CKS substrate. The architectural commitment is at the substrate-as-coordination-artifact level, not at the external-memory-presence level.

6. Failure modes that violate the boundary

Seven failure modes name ways an implementation can fail by treating CKS as a parametric pattern or by conflating substrate and parametric memory.

Substrate-content-in-weights. Substrate is treated as content to be incorporated into the LLM's weights through fine-tuning, with the model treating its weights as authoritative for substrate content. The commitment to external substrate per A1.02 fails, and Property C per A2.21 fails because substrate-relevant content is in weights.

Parametric-style cost-curve assumptions. The implementation imports parametric expansion's cost-curve assumptions (superlinear, frontier-bending) into CKS expectations. Optimization decisions, scaling plans, and capacity provisioning are based on parametric assumptions; the linear-cost commitment per A1.06 operates on different dynamics than the operators expect.

Training-as-substrate-update. Knowledge editing (selective weight modification) or continual learning (progressive weight updates) is treated as equivalent to substrate updates. Substrate's architectural commitments — A2.08 layer commitments, provenance per A2.40, governance per A1.01, determinism per A2.57 — are absent because the "updates" happen in weights, not in substrate.

Parametric-memory-as-substrate-substitute (including derived and hidden variants). The implementation uses parametric memory as a substitute for or shadow of substrate: fine-tuning the LLM on coordination content that A1.13 specifies must be in substrate; deriving substrate content from the LLM's parametric memory by automated extraction, bypassing human authorship per A2.66; or claiming substrate is external while operationally fine-tuning the LLM on substrate content periodically. In all variants the commitment to substrate-as-coordination-artifact is nominally present but operationally violated.

Mixed-storage architecture without architectural distinction. The implementation has both substrate and parametric memory but does not architecturally distinguish which content lives where. Coordination content drifts between substrate and weights; the substrate-cell boundary per A1.02 becomes operationally ambiguous, and the layer-naming discipline A2.85 requires for hybrid composition coherence is absent.

Parametric-recall-as-substrate-read. The implementation operates substrate reads through parametric recall — the LLM "remembers" substrate content from training exposure rather than reading from external storage at operation time. Guarantee A per A2.57 fails because parametric recall is non-deterministic, and the substrate-only-path commitment per A2.41 fails because the read does not actually traverse the substrate.

Provenance erasure through parametric expansion. The implementation expands its knowledge through training procedures and treats the resulting weight-encoded content as functionally equivalent to substrate content. The six provenance fields per A2.40 have no parametric equivalent — gradient updates do not produce field-addressable provenance — so retraceability per A1.07 fails silently. Audits that depend on provenance return absences, not records.

7. Operational test, and why naming the boundary matters

A system instantiates the CKS-vs-parametric-memory boundary if and only if all of the following are true at all times during the substrate's existence:

1. The substrate is external to the LLM per component (a) of §3 — substrate persists in storage architecture distinct from the LLM's weights.
2. Substrate state changes through human-governed writes per component (b) of §3 — content is added by human authorship, by LLM drafting under orchestration rules, or by stable cells operating under rules; never by gradient updates to the model.
3. Substrate is recovered through deterministic reads per component (c) of §3 — reads against substrate return the same content deterministically per Guarantee A per A2.57.
4. Substrate expansion is linear-in-additions per component (d) of §3 — storage cost scales linearly with content per A2.30; processing cost scales with operations per A2.32.
5. The LLM does not hold substrate-relevant state outside substrate per Property C per A2.21 — the LLM's weights do not carry coordination content equivalent to substrate.
6. Hybrid compositions with fine-tuned mediators (per A1.16 and A2.85) explicitly name the layers — the parametric layer in the mediator improves fluency; the CKS layer carries coordination decisions and rationale.

A system that fails any of (1)–(6) does not instantiate the boundary in the architectural sense, even if it operationally appears to combine substrate-style and parametric-style features.

The one-sentence test. *If expanding the system's knowledge requires modifying model weights through any training procedure, the memory is parametric; if expansion happens through writes to an addressable external store under human authority, it is not.* The one-sentence test names the most operationally distinctive axis (expansion mechanism) and is useful for analysts and reviewers who need to classify a specific architecture quickly. The four-component specification per §3 provides the full architectural definition for cases requiring detailed analysis.

Why naming the boundary as standalone matters. Implementations under pressure to deliver "AI memory" capabilities consistently drift toward parametric framings. The drift is steady because parametric memory is the foundational LLM concern — audiences understand "the model knows X" more readily than "the substrate carries X with provenance and authority" — and because fine-tuning is operationally familiar from extensive prior work. Implementations that drift away from the boundary produce systems where coordination content drifts into the LLM's weights, with concrete consequences: cost-curve failures (A1.06), determinism failures (A2.57), governance failures (A1.01), provenance failures (A2.40), and architectural-commitment failures across A1.02, A1.04, and A1.06 simultaneously.

Naming the boundary as a standalone architectural commitment — with the four operational components in §3, the limitations in §4, the four adjacent-pattern distinctions in §5, the failure modes in §6, and the operational test in this section — gives downstream readers a precise specification of what distinguishes CKS from parametric memory. Subsequent notes A2.84 and A2.85 specialize Adjacency 3 (KO/OIDA) and hybrid composition coherence respectively, and together close the decomposition of A1.14.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Related notes in this series

Li, W. (2026). *Authority, Not Labor: A Precise Definition of "Human-Governed" in the Coordination Knowledge Substrate Pattern*. 24 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Three Adjacencies: Why CKS Is Not RAG, Not Parametric Memory, and Not External Structured Memory*. 24 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Adjacency 2 — CKS Is Not Parametric Memory: Standalone Treatment of the Architectural Boundary Between CKS and In-Weight Memory*. 5 May 2026. ORCID: 0009-0004-8065-3235.